

使用 WebXR Device API 建構擴增實境 (AR) 應用程式

來源：<https://codelabs.developers.google.com/ar-with-webxr?hl=zh-tw>

步驟數：6

瞭解如何將擴增實境功能與 WebXR Device API 搭配使用，以及運用場景理解技術，將 3D 物體放置在現實世界中。

目錄

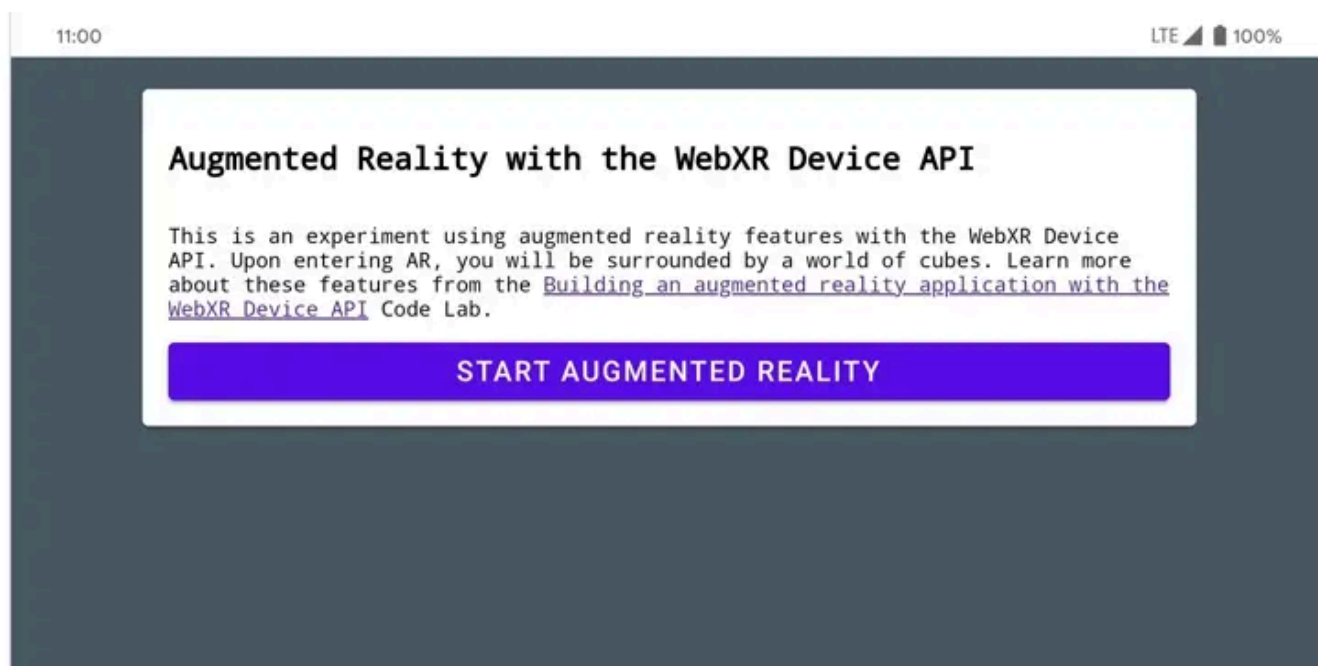
1. [1. 事前準備](#)
2. [2. 設定開發環境](#)
3. [3. 設定 WebXR](#)
4. [4. 新增瞄準線](#)
5. [5. 新增陰影](#)
6. [6. 其他資源](#)

步驟 1 · 約 2 分鐘

1. 事前準備

本程式碼研究室會逐步說明如何建構 AR 網頁應用程式，並使用 JavaScript 算繪 3D 模型，讓模型看起來像是存在於現實世界。

您可以使用結合擴增實境和虛擬實境 (VR) 功能的 [WebXR Device API](#)。您將著重於 WebXR Device API 的 AR 擴充功能，建立可在互動式網路上執行的簡易 AR 應用程式。



什麼是 AR？

AR 通常是指將電腦生成的圖像與現實世界混合。如果是手機 AR，則是指在即時攝影機畫面中，逼真地放置電腦繪圖。為了讓這項效果在手機移動時保持真實感，支援 AR 的裝置必須瞭解移動時所處的世界，並判斷在 3D 空間中的姿勢 (位置和方向)。這可能包括偵測表面和估算環境照明。

自從 Google 發布 [ARCore](#) 和 Apple 發布 [ARKit](#) 後，AR 技術已廣泛應用於各種應用程式，無論是自拍濾鏡還是 AR 遊戲，都能看到 AR 技術的身影。

建構項目

在本程式碼研究室中，您將建構網頁應用程式，透過擴增實境將模型放置在現實世界中。您的應用程式將會：

1. 使用目標裝置的感應器，判斷及追蹤裝置在世界中的位置和方向
2. 在攝影機即時畫面頂端算繪合成的 3D 模型
3. 執行命中測試，將物件放置在真實世界中探索到的表面上

課程內容

- 如何使用 [WebXR 裝置 API](#)
- 如何設定基本 AR 場景
- 如何使用 AR 命中測試尋找表面
- 如何載入及算繪與真實世界攝影機影像同步的 3D 模型
- 如何根據 3D 模型算繪陰影

本程式碼研究室著重於 AR API。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上對應存放區程式碼即可。

軟硬體需求

- 用於編碼和託管靜態網頁內容的工作站
- [支援 ARCore 的 Android 裝置](#)，搭載 [Android 8.0 Oreo](#)
- [Google Chrome](#)
- 已安裝 [Google Play 服務 - AR 適用](#) (Chrome 會在相容裝置上自動提示您安裝)
- 選擇的網路伺服器
- USB 傳輸線，用於將 AR 裝置連接至工作站
- [範例程式碼](#)
- 文字編輯器
- 具備 HTML、CSS、JavaScript 和 [Google Chrome 開發人員工具](#) 的基本知識

在 AR 裝置上按一下「Try it」，即可試用本程式碼研究室的第一個步驟。如果頁面顯示「您的瀏覽器沒有 AR 功能」訊息，請確認 Android 裝置已安裝「Google Play 服務 - AR 適用」。

步驟 2 · 約 4 分鐘

2. 設定開發環境

下載程式碼

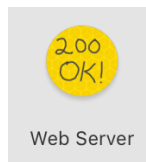
1. 點選下方連結，即可在工作站下載這個程式碼研究室的所有程式碼：
2. 將下載的 ZIP 檔案解壓縮。這會解壓縮根資料夾 (`ar-with-webxr-master`)，其中包含本程式碼研究室多個步驟的目錄，以及您需要的所有資源。

「 `step-03` 」和「 `step-04` 」資料夾包含本程式碼研究室第三和第四個步驟的預期最終狀態，以及 `final` 結果。僅供參考。

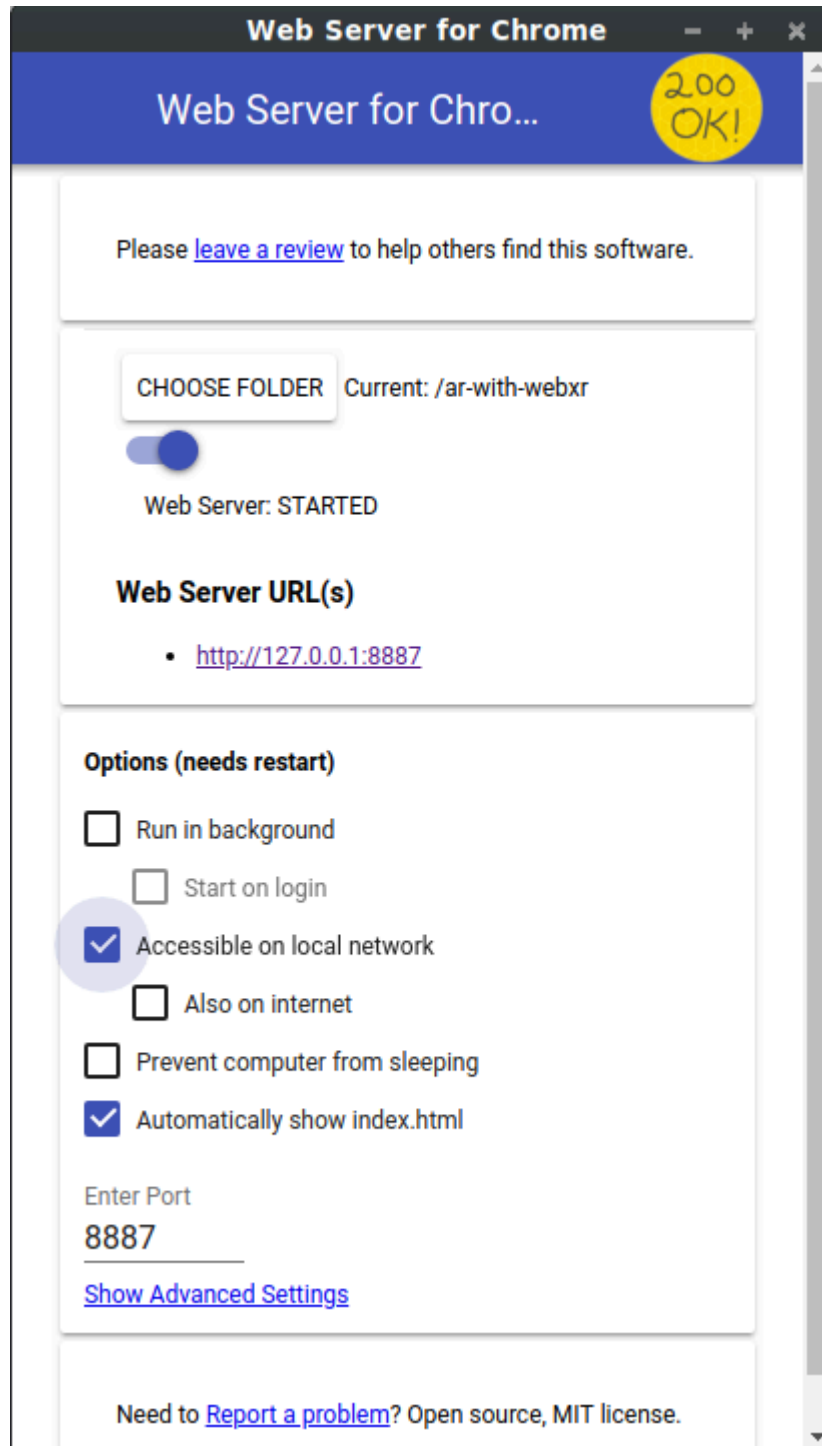
您會在 `work` 目錄中完成所有編碼工作。

安裝網路伺服器

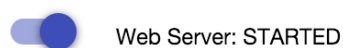
1. 您可以自由使用自己的網路伺服器。如果尚未設定，本節將詳細說明如何設定 Web Server for Chrome。
如果工作站尚未安裝該應用程式，可以從 Chrome 線上應用程式商店安裝。
2. 安裝 Web Server for Chrome 應用程式後，請前往 `chrome://apps` 並點選「Web Server」圖示：



接著會看到這個對話方塊，可供您設定本機網路伺服器：



3. 按一下「選擇資料夾」，然後選取 `ar-with-webxr-master` 資料夾。這樣一來，您就能透過網頁伺服器對話方塊中醒目標顯示的網址 (位於「Web Server URL(s)」部分)，放送進行中的工作。
4. 在「選項 (需要重新啟動)」下方，勾選「自動顯示 index.html」核取方塊。
5. 將「Web server」切換為「Stop」，然後切換回「Started」。



6. 確認至少顯示一個網頁伺服器網址：`http://127.0.0.1:8887`，這是預設的本機主機網址。

設定通訊埠轉送

設定 AR 裝置，讓裝置在存取 `localhost:8887` 時，能存取工作站上的相同通訊埠。

1. 在開發工作站上前往 `chrome://inspect`，然後按一下「通訊埠轉送...」：

Devices

☒ Discover USB devices

Port forwarding...

☒ Discover network targets

Configure...

[Open dedicated DevTools for Node](#)

2. 使用「通訊埠轉送設定」對話方塊，將通訊埠 8887 轉送至 `localhost:8887`。

3. 勾選「啟用通訊埠轉送」核取方塊：

Port forwarding settings

8080	localhost:8080
8887	localhost:8887

Port

IP address and port

Define the listening port on your device that maps to a port accessible from your development machine. [Learn more](#)

☒ Enable port forwarding

Done

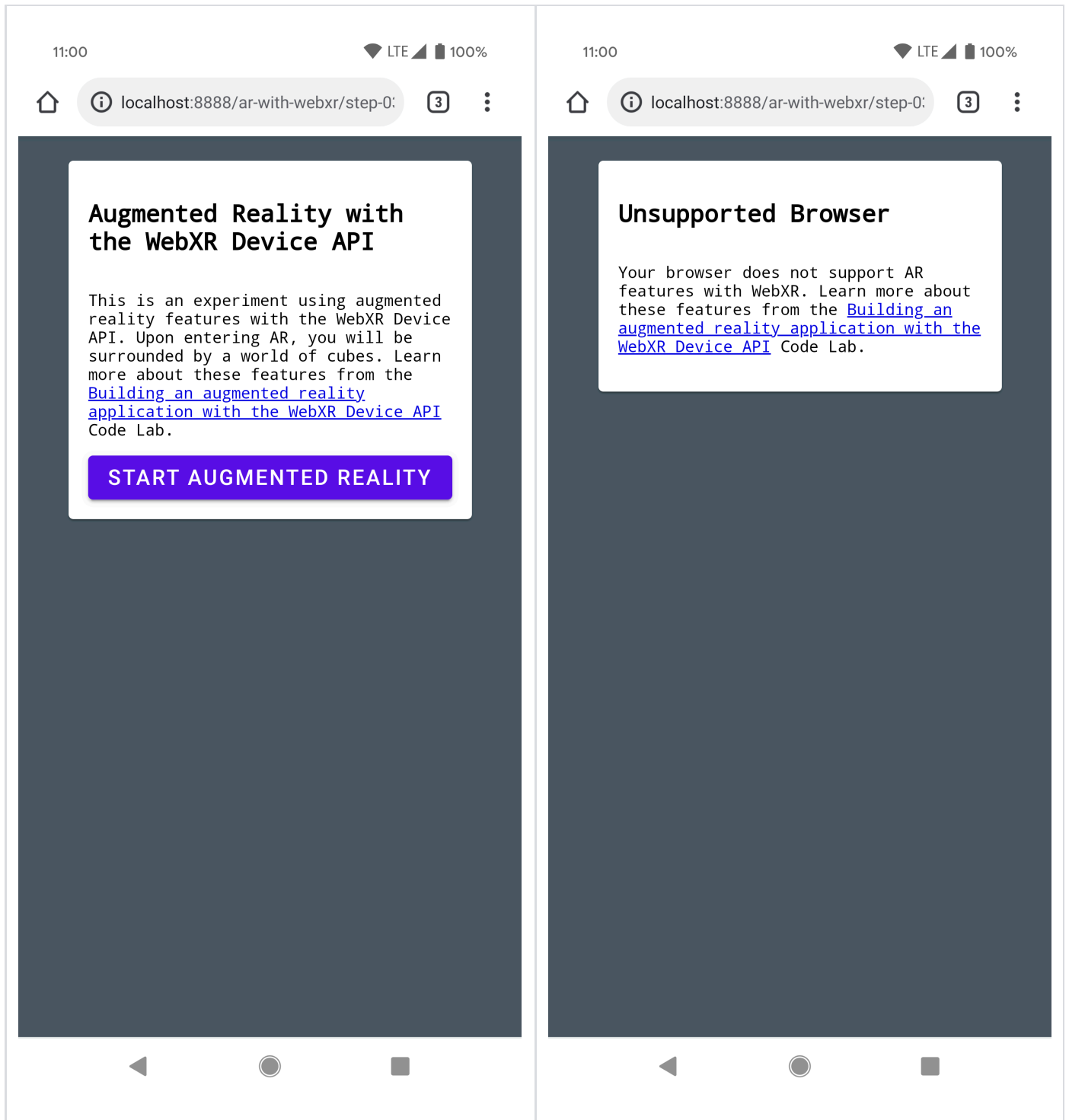
驗證設定

測試連線：

1. 使用 USB 傳輸線將 AR 裝置連接至工作站。
2. 在 Chrome 的 AR 裝置上，於網址列中輸入 `http://localhost:8887`。AR 裝置應將這項要求轉送至開發工作站的網路伺服器。畫面上應會顯示檔案目錄。
3. 在 AR 裝置上，按一下 `step-03`，在瀏覽器中載入 `step-03/index.html` 檔案。

畫面上應會顯示「啟動擴增實境」按鈕

不過，如果看到「不支援的瀏覽器」錯誤頁面，表示裝置可能不相容。



現在，AR 裝置應該可以連線至網路伺服器。

4. 按一下「啟動擴增實境」。系統可能會提示您安裝 ARCore。

 To view augmented reality content, install ARCore 

INSTALL

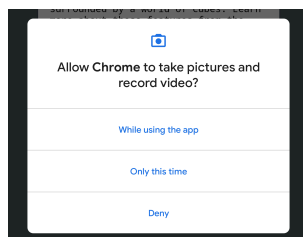
首次執行 AR 應用程式時，系統會提示您授予相機權限。

⚠ Chrome needs permission to access your camera to create a 3D map of your surroundings ✕

Continue



→



一切就緒後，您應該會看到攝影機畫面疊加了方塊場景。相機剖析的世界越多，場景理解能力就越好，因此四處移動有助於穩定畫面。



重要事項：基於安全考量，WebXR Device API 只能在安全 (HTTPS) 環境中執行，但本機開發作業除外。如果無法啟用 WebXR，請確認您使用的是安全文件或 localhost 網址。

從現在開始，所有測試和驗證 (後續步驟中的「試試看」部分) 都需要透過 AR 裝置造訪連結。

步驟 3 · 約 5 分鐘

3. 設定 WebXR

在本步驟中，您將瞭解如何設定 WebXR 工作階段和基本 AR 場景。HTML 網頁會提供 CSS 樣式和 JavaScript，以啟用基本 AR 功能。這可加快設定程序，讓程式碼研究室專注於 AR 功能。

HTML 網頁

您可以使用現有的網路技術，在傳統網頁中建構 AR 體驗。在這個體驗中，您會使用全螢幕的算繪畫布，因此 HTML 檔案不必太複雜。

AR 功能需要使用者手勢才能啟動，因此您可以使用一些 [Material Design 元件](#)，顯示「啟動 AR」按鈕和不支援的瀏覽器訊息。

`work` 目錄中現有的 `index.html` 檔案應如下所示。這是實際內容的子集，請勿將此程式碼複製到檔案中！

```
<!-- Don't copy this code into your file! -->
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Building an augmented reality application with the WebXR Device API</title>
    <link rel="stylesheet" href="https://unpkg.com/material-components-
web@latest/dist/material-components-web.min.css">
    <script src="https://unpkg.com/material-components-web@latest/dist/material-components-
web.min.js"></script>

    <!-- three.js -->
    <script src="https://unpkg.com/three@0.123.0/build/three.js"></script>
    <script src="https://unpkg.com/three@0.123.0/examples/js/loaders/GLTFLoader.js"></script>

    <script src="../../shared/utils.js"></script>
    <script src="app.js"></script>
  </head>
  <body>
    <!-- Information about AR removed for brevity. -->

    <!-- Starting an immersive WebXR session requires user interaction. Start the WebXR
experience with a simple button. -->
    <a onclick="activateXR()" class="mdc-button mdc-button--raised mdc-button--accent">
      Start augmented reality
    </a>

  </body>
</html>
```

開啟主要 JavaScript 程式碼

應用程式的起點位於 `app.js`。這個檔案提供一些樣板，用於設定 AR 體驗。

這個樣板使用非同步函式。如要進一步瞭解非同步函式，請參閱「[非同步函式 - 讓 Promise 更容易使用](#)」。

您的工作目錄也已包含應用程式程式碼 (`app.js`)。

查看 WebXR 和 AR 支援情形

使用者處理 AR 前，請先檢查 `navigator.xr` 和必要的 XR 功能是否存在。`navigator.xr` 物件是 WebXR Device API 的進入點，因此如果裝置相容，這個物件就應該存在。此外，請確認系統支援 `"immersive-ar"` 工作階段模式。

如果一切正常，按一下「進入擴增實境」按鈕，系統就會嘗試建立 XR 工作階段。否則，系統會呼叫 `onNoXRDevice()` (在 `shared/utils.js` 中)，顯示指出缺少 AR 支援的訊息。

`app.js` 中已有這段程式碼，因此無須變更。

```
(async function() {
  if (navigator.xr && await navigator.xr.isSessionSupported("immersive-ar")) {
    document.getElementById("enter-ar").addEventListener("click", activateXR)
  } else {
    onNoXRDevice();
  }
})();
```

即使瀏覽器支援 WebXR 裝置 API，也可能不支援所要求的會話模式。舉例來說，電腦瀏覽器可能會實作 API，但沒有任何已連線的 VR 或 AR 硬體來支援體驗。如要進一步瞭解裝置列舉，請參閱 [WebXR 裝置 API 規格](#)。

要求 `XRSession`

點選「Enter augmented Reality」(進入擴增實境) 時，程式碼會呼叫 `activateXR()`。即可啟動 AR 體驗。

1. 在 `app.js` 中找出 `activateXR()` 函式。部分程式碼已省略：

```
activateXR = async () => {
  // Initialize a WebXR session using "immersive-ar".
  this.xrSession = /* TODO */;

  // Omitted for brevity
}
```

WebXR 的進入點是 `XRSystem.requestSession()`。使用 `immersive-ar` 模式，即可在真實環境中查看算繪內容。

2. 使用 `"immersive-ar"` 模式初始化 `this.xrSession`：

```
activateXR = async () => {  
  // Initialize a WebXR session using "immersive-ar".  
  this.xrSession = await navigator.xr.requestSession("immersive-ar");  
  
  // ...  
}
```

初始化 `XRReferenceSpace`

`XRReferenceSpace` 說明虛擬世界中物件使用的座標系統。'local' 模式最適合用於 AR 體驗，參考空間的原點位於檢視者附近，且追蹤穩定。

在 `onSessionStarted()` 中使用下列程式碼初始化 `this.localReferenceSpace`：

```
this.localReferenceSpace = await this.xrSession.requestReferenceSpace("local");
```

定義動畫迴圈

1. 使用 `XRSession` 的 `requestAnimationFrame` 啟動算繪迴圈，與 `window.requestAnimationFrame` 類似。

`XRSession` 的 `requestAnimationFrame` 可讓您掛接原生 XR 裝置的刷新率。標準網頁的算繪迴圈是為 60 FPS 設計，但外部 VR 螢幕可能會以 120 FPS 算繪。非專屬 AR 工作階段可能仍只會以 60 FPS 執行，但裝置的姿勢和檢視資訊只能在工作階段的 `requestAnimationFrame` 中存取。

系統會在每個影格呼叫 `onXRFrame`，並提供時間戳記和 `XRFrame`。

2. 完成 `onXRFrame` 的實作。繪製影格時，請新增下列項目，將下一個要求加入佇列：

```
// Queue up the next draw request.  
this.xrSession.requestAnimationFrame(this.onXRFrame);
```

3. 新增程式碼以設定圖形環境。在 `onXRFrame` 底部新增：

```
// Bind the graphics framebuffer to the baseLayer's framebuffer.  
const framebuffer = this.xrSession.renderState.baseLayer.framebuffer;  
this.gl.bindFramebuffer(this.gl.FRAMEBUFFER, framebuffer);  
this.renderer.setFramebuffer(framebuffer);
```

4. 如要判斷觀看者姿勢，請使用 `XRFrame.getViewerPose()`。這項 `XRViewerPose` 描述裝置在空間中的位置和方向。其中也包含 `XRView` 陣列，說明場景應從每個視角算繪，才能在目前裝置上正確顯示。立體虛擬實境有兩個檢視畫面 (每隻眼睛各一個)，但 AR 裝置只有一個檢視畫面。

`pose.views` 中的資訊最常用於設定虛擬攝影機的檢視矩陣和投影矩陣。這會影響 3D 場景的布局。設定攝影機後，即可算繪場景。

5. 在 `onXRFrame` 底部新增：

```
// Retrieve the pose of the device.
// XRFrame.getViewerPose can return null while the session attempts to establish tracking.
const pose = frame.getViewerPose(this.localReferenceSpace);
if (pose) {
  // In mobile AR, we only have one view.
  const view = pose.views[0];

  const viewport = this.xrSession.renderState.baseLayer.getViewport(view);
  this.renderer.setSize(viewport.width, viewport.height);

  // Use the view's transform matrix and projection matrix to configure the THREE.camera.
  this.camera.matrix.fromArray(view.transform.matrix);
  this.camera.projectionMatrix.fromArray(view.projectionMatrix);
  this.camera.updateMatrixWorld(true);

  // Render the scene with THREE.WebGLRenderer.
  this.renderer.render(this.scene, this.camera);
}
```

測試

執行應用程式，並在開發裝置上前往 `work/index.html`。畫面上應該會顯示攝影機畫面，以及在空間中漂浮的立方體，且立方體的透視角度會隨著裝置移動而改變。追蹤效果會隨著你的移動而提升，因此請找出最適合你和裝置的追蹤方式。



如果執行應用程式時發生問題，請參閱「[簡介](#)」和「[設定開發環境](#)」章節。

步驟 4 · 約 5 分鐘

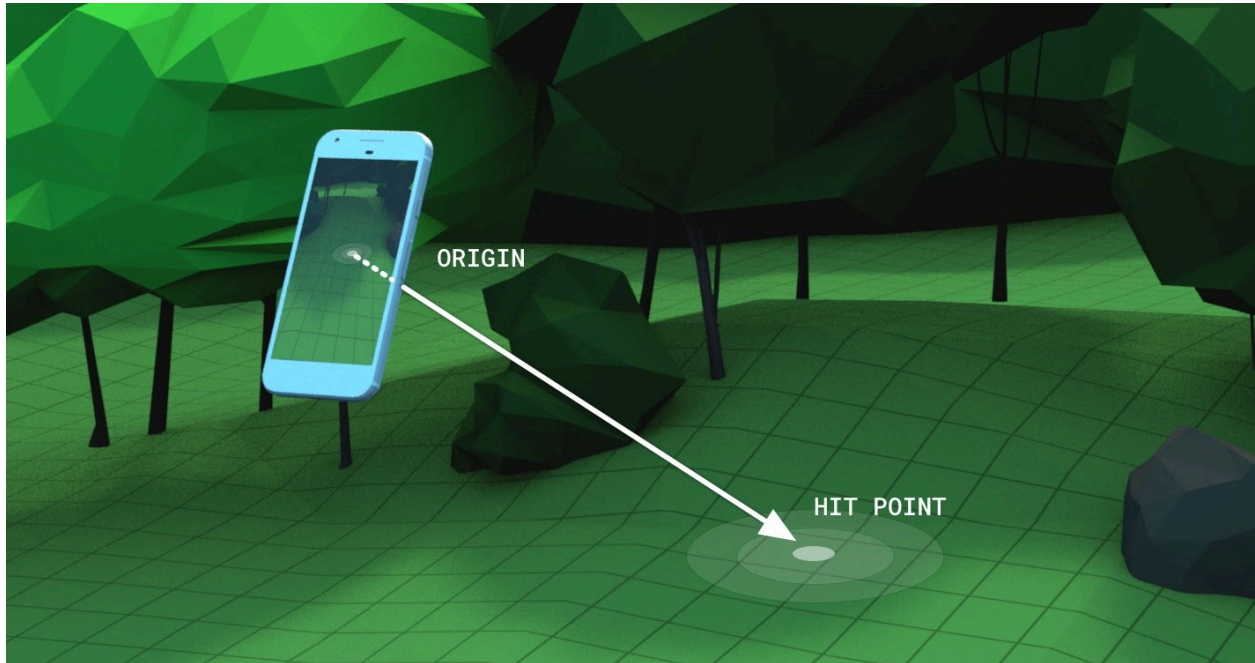
4. 新增瞄準線

設定基本 AR 場景後，就可以開始使用命中測試與現實世界互動。在本節中，您將編寫命中測試程式，並使用該程式尋找現實世界中的表面。

瞭解命中測試

一般來說，命中測試是指從空間中的某個點沿某個方向投射直線，並判斷是否與任何感興趣的物件相交。在本例中，您將裝置對準現實世界中的某個位置。想像從裝置鏡頭發出的光線，直接射入前方的實體世界。

WebXR Device API 可讓您瞭解這道光線是否與現實世界中的任何物件相交，這取決於基礎 AR 功能和對世界的理解。



要求提供 `XRSession`，並加購額外功能

如要進行熱門測試，要求 `XRSession` 時必須使用額外功能。

1. 在 `app.js` 中找出 `navigator.xr.requestSession`。
2. 新增 `"hit-test"` 和 `"dom-overlay"` 功能，如下所示：`requiredFeature`

```
this.xrSession = await navigator.xr.requestSession("immersive-ar", {
  requiredFeatures: ["hit-test", "dom-overlay"]
});
```

3. 設定 DOM 疊加層。將 `document.body` 元素疊加在 AR 攝影機畫面上，如下所示：

```
this.xrSession = await navigator.xr.requestSession("immersive-ar", {
  requiredFeatures: ["hit-test", "dom-overlay"],
  domOverlay: { root: document.body }
});
```

新增動作提示

ARCore 充分瞭解環境後，就能發揮最佳效果。這項技術稱為「即時定位與地圖建構」(SLAM)，會使用視覺上明顯不同的特徵點，計算位置和環境特徵的變化。

使用上一個步驟中的 `"dom-overlay"`，在攝影機串流畫面上方顯示動作提示。

將 `<div>` 新增至 ID 為 `stabilization` 的 `index.html`。這個 `<div>` 會向使用者顯示動畫，代表穩定狀態，並提示他們移動裝置，以強化 SLAM 程序。使用者進入 AR 模式後，系統會顯示這個圖示，並在十字線找到表面後隱藏，由 `<body>` 類別控制。

```
<div id="stabilization"></div>

</body>
</html>
```

新增十字線

使用十字線指出裝置視野所指的位置。

1. 在 `app.js` 中，將 `setupThreeJs()` 中的 `DemoUtils.createCubeScene()` 呼叫替換為空白的 `Three.Scene()`。

```
setupThreeJs() {
  // ...

  // this.scene = DemoUtils.createCubeScene();
  this.scene = DemoUtils.createLitScene();
}
```

2. 在新的場景中填入代表碰撞點的物件。提供的 `Reticle` 類別會處理 `shared/utils.js` 中的十字線模型載入作業。
3. 在 `setupThreeJs()` 中將 `Reticle` 新增至場景：

```
setupThreeJs() {
  // ...

  // this.scene = DemoUtils.createCubeScene();
  this.scene = DemoUtils.createLitScene();
  this.reticl = new Reticle();
  this.scene.add(this.reticl);
}
```

如要執行命中測試，請使用新的 `XRReferenceSpace`。這個參考空間會從觀看者的角度指出新的座標系統，以建立與觀看方向對齊的光線。`XRSession.requestHitTestSource()` 會使用這個座標系統，計算命中測試。

4. 在 `app.js` 的 `onSessionStarted()` 中新增下列內容：

```

async onSessionStarted() {
  // ...

  // Setup an XRReferenceSpace using the "local" coordinate system.
  this.localReferenceSpace = await this.xrSession.requestReferenceSpace("local");

  // Add these lines:
  // Create another XRReferenceSpace that has the viewer as the origin.
  this.viewerSpace = await this.xrSession.requestReferenceSpace("viewer");
  // Perform hit testing using the viewer as origin.
  this.hitTestSource = await this.xrSession.requestHitTestSource({ space: this.viewerSpace
});

  // ...
}

```

5. 使用這個 `hitTestSource`，對每個影格執行命中測試：

- 如果點擊測試沒有結果，表示 ARCore 尚未有足夠時間瞭解環境。在這種情況下，請使用穩定性 `<div>` 提示使用者移動裝置。
- 如有結果，請將十字線移至該位置。

6. 修改 `onXRFrame` 即可移動十字線：

```

onXRFrame = (time, frame) => {
  // ... some code omitted ...
  this.camera.updateMatrixWorld(true);

  // Add the following:
  const hitTestResults = frame.getHitTestResults(this.hitTestSource);

  if (!this.stabilized && hitTestResults.length > 0) {
    this.stabilized = true;
    document.body.classList.add("stabilized");
  }
  if (hitTestResults.length > 0) {
    const hitPose = hitTestResults[0].getPose(this.localReferenceSpace);

    // update the reticle position
    this.reticle.visible = true;
    this.reticle.position.set(hitPose.transform.position.x, hitPose.transform.position.y,
hitPose.transform.position.z)
    this.reticle.updateMatrixWorld(true);
  }
  // More code omitted.
}

```



新增輕觸螢幕時的行為

`XRSession` 可以透過 `select` 事件 (代表主要動作) 根據使用者互動發出事件。在行動裝置上使用 WebXR 時，主要動作是輕觸螢幕。

1. 在 `onSessionStarted` 底部新增 `select` 事件監聽器：

```
this.xrSession.addEventListener("select", this.onSelect);
```

在這個範例中，輕觸螢幕會在十字線位置放置向日葵。

2. 在 `App` 類別中建立 `onSelect` 的實作：

```
onSelect = () => {  
  if (window.sunflower) {  
    const clone = window.sunflower.clone();  
    clone.position.copy(this.reticule.position);  
    this.scene.add(clone);  
  }  
}
```

測試應用程式

您已建立十字線，可使用裝置透過命中測試瞄準。輕觸螢幕後，您應該就能在十字線指定的位置放置向日葵。

1. 執行應用程式時，您應該會看到十字線追蹤地板表面。如果沒有，請慢慢移動手機環顧四周。
2. 看到十字線後，輕觸十字線。並在上面放上向日葵。您可能需要稍微移動，讓底層的 AR 平台更準確地偵測現實世界中的表面。低光源環境和沒有特徵的表面會降低場景理解品質，並增加找不到命中點的機率。如果遇到任何問題，請查看 `step-04/app.js` 程式碼，瞭解這個步驟的運作範例。



5. 新增陰影

如要建立逼真的場景，數位物件必須有適當的光線和陰影，才能讓場景更逼真，帶來身歷其境的感受。

`three.js` 會處理光線和陰影。您可以指定哪些燈光應投射陰影、哪些材質應接收及算繪這些陰影，以及哪些網格可以投射陰影。這個應用程式的場景包含投射陰影的光線，以及僅用於算繪陰影的平面。

1. 在 `three.js` `WebGLRenderer` 上啟用陰影。建立算繪器後，請在其 `shadowMap` 上設定下列值：

```
setupThreeJs() {  
  ...  
  this.renderer = new THREE.WebGLRenderer(...);  
  ...  
  this.renderer.shadowMap.enabled = true;  
  this.renderer.shadowMap.type = THREE.PCFSoftShadowMap;  
  ...  
}
```

本範例使用陰影的硬式編碼光源位置。日後，WebXR 的 AR 功能可能會加入光線估算和其他值，讓 3D 模型的陰影也能與現實世界同步。如需最新資訊，請參閱「[功能：WebXR AR 燈光估算](#)」。

在 `DemoUtils.createLitScene()` 中建立的範例場景包含名為 `shadowMesh` 的物件，這是只會算繪陰影的水平平面。這個介面一開始的 Y 位置為 10,000 個單位。放置向日葵後，請移動 `shadowMesh`，使其與現實世界的表面高度相同，這樣花朵的陰影就會顯示在現實世界的地面上。

2. 在 `onSelect` 中，將 `clone` 新增至場景後，請加入程式碼來重新放置陰影平面：

```
onSelect = () => {  
  if (window.sunflower) {  
    const clone = window.sunflower.clone();  
    clone.position.copy(this.reticle.position);  
    this.scene.add(clone);  
  
    const shadowMesh = this.scene.children.find(c => c.name === "shadowMesh");  
    shadowMesh.position.y = clone.position.y;  
  }  
}
```

您可能需要運用更多場景理解功能，才能顯示更複雜的場景和準確的陰影。

測試

放置向日葵時，您應該會看到向日葵投射陰影。如果遇到任何問題，請查看 `final/app.js` 程式碼，瞭解這個步驟的運作範例。



步驟 6 · 約 2 分鐘

6. 其他資源

恭喜！您已完成本程式碼研究室，瞭解如何使用 WebXR 建立 AR 體驗。

瞭解詳情

- [WebXR 裝置 API 規格](#)
 - [MDN 上的 WebXR 裝置 API](#)
 - [WebXR 範例](#)
 - [WebXR 的命中測試規格](#)
 - [WebXR 的錨點提案](#)
 - [three.js](#)
-